

M3 · Lab Parte 2

# Implementación + Verificación

Pasos 3 y 4 del TDD · con auditoria grupal

---

Martes 11-ago-2026 · 19:00-21:00 · Instituto CIRD



## La sesión de hoy · 4 bloques

19:00 · 19:25

**Q&A + Alineación**

Timeline · evaluaciones M3 · rama Lab 2 · dudas del sprint

19:25 · 19:50

**Auditoría de tests · en pares**

Los 6 riesgos con ejemplos reales del Escenario A

19:50 · 20:50

**Lab en vivo · Pasos 3 y 4**

Implementación + Verificación con TDD asistido

20:50 · 21:00

**Cierre M3 · gobernanza de tests**

3 principios que cierran el M3 y siembran M5



## Timeline vigente · estamos donde debemos estar

|                  |                                    |
|------------------|------------------------------------|
| HOY · mar 11-ago | Lab Parte 2 (Pasos 3-4)            |
| jue 13-ago       | Arranque M4 · Agentes              |
| dom 17-ago 23:59 | Entrega Lab 3 + Cuestionario M3    |
| 18-24-ago        | M4                                 |
| lun 25-ago       | Abre #demo-day-2026-I · arranca M5 |
| lun 31-ago 23:59 | Cierre absoluto                    |



## Evaluaciones M3 · cómo se califica

### Peso M3

#### Lab 3 · 50%

Endpoint TDD · rúbrica 6 indicadores

#### Cuestionario M3 · 50%

Autocorrección en formulario · selección múltiple

**Ambos cierran dom 17-ago 23:59**

### Cómo arrancar tu rama Lab 3

- Si mergeaste Lab 2 a main → git checkout -b lab3
- Si tenés Lab 2 sin mergear → git checkout lab2 && git checkout -b lab3
- Si no cerraste Lab 2 → copió los 4 artefactos del repositorio del curso



### Preguntas del cohorte · abrimos micrófono

¿Cómo va tu Lab 3?  
¿Llegaste al Paso 2 (tests en rojo)?  
¿Hay algo del PRD que te está frenando?

## 💡 Regla de oro del testing con IA

***El verde de un test es una hipótesis · no una prueba.***

La calidad del verde depende 100% de la calidad del assert.  
Si el assert es débil · si compara con un valor inventado · si atrapa cualquier excepción  
· el verde es cosmético.

El TDD con IA suma un paso al ciclo original:

**Rojo → Verde → AUDITORÍA → Refactor**

*Sin auditoría, la IA nos regala falsos positivos que pasan por robustez.*

## Los 6 riesgos + las 5 preguntas que los cazan

### 6 RIESGOS

- 1 · Oráculo inventado
- 2 · Test tautológico
- 3 · Mock excesivo
- 4 · Happy path dominante
- 5 · API alucinada
- 6 · Falso verde

### 5 PREGUNTAS DE AUDITORÍA

- P1 · ¿El **assert** compara con **valor real** o con lo que **el mock** devuelve?
- P2 · ¿El test fallaría **si la implementación estuviera mal**?
- P3 · ¿Cubre solo **happy path** o también **borde/error**?
- P4 · ¿Los **métodos** usados **existen** realmente en la librería?
- P5 · ¿El **verde** es por lógica real o por trivialidad?

## Ejercicio · Auditoría de tests

1. ¿Qué riesgo tiene?
2. ¿Qué pregunta lo detecta?
3. ¿Cómo lo corregirías?

### Ejemplo · Par 1

```
def test_historial_devuelve_transacciones():
    client = TestClient(app)
    headers = {"Authorization": "Bearer valid_jwt"}
    response = client.get(
        "/api/v1/transacciones?desde=2026-07-01&hasta=2026-08-01",
        headers=headers,
    )
    assert response.json()["data"][0]["monto"] == 1500.00
    assert response.json()["data"][0]["comercio_nombre"] == "Comercio X"
```



## Oráculo Inventado

### Pregunta

### Señal en el código

P1 · P2

Números o strings hardcoded sin fixture

```
def test_historial_devuelve_transacciones():
    # Arrange
    client = TestClient(app)
    headers = {"Authorization": "Bearer valid_jwt"}

    # Act
    response = client.get(
        "/api/v1/transacciones?
        desde=2026-07-01&hasta=2026-08-01",
        headers=headers,
    )

    # Assert
    assert response.json()["data"][0]["monto"] == 1500.00 # ⚠️
    ¿de dónde salió 1500?
    assert response.json()["data"][0]["comercio_nombre"] ==
    "Comercio X" # ⚠️ el PRD no define este campo
```

**P1:** el `1500.00` no está anclado a nada · ni fixture · ni cálculo · ni referencia al PRD

**P2:** si la implementación cambia el monto, este assert falla · pero no porque la lógica esté mal · sino porque el número es arbitrario

"Los asserts deben validar el formato de la respuesta según el PRD. Sin valores literales inventados. Si necesitás datos, definilos como fixtures explícitos."

```
assert response.status_code == 200
body = response.json()
assert "data" in body
assert "pagination" in body
assert "next_cursor" in body["pagination"]
assert "has_more" in body["pagination"]
# Cada item respeta el contrato Pydantic
if body["data"]:
    assert set(body["data"][0].keys()) >= {"id",
    "created_at", "monto", "estado"}
```





## Test tautológico

| Pregunta | Señal en el código               |
|----------|----------------------------------|
| P2 · P5  | <code>assert f(x) == f(x)</code> |

```
def test_calcular_next_cursor():  
    # Arrange  
    ultima_transaccion = {"created_at": "2026-08-01T12:00:00",  
                           "id": "uuid-123"}  
  
    # Act  
    cursor = calcular_cursor(ultima_transaccion)  
  
    # Assert  
    assert calcular_cursor(ultima_transaccion) == cursor # ⚠  
    compara la función consigo misma
```

"Los asserts deben comparar el output  
contra un valor esperado  
independiente, nunca contra otra  
llamada a la misma función."

**P2:** la implementación puede estar 100% rota · el test  
sigue verde porque compara el output con el mismo  
output

**P5:** verde trivial · no valida comportamiento

```
# Assert · verifica propiedades REALES del cursor (según  
ADR-0003)  
import base64, json  
decoded = json.loads(base64.b64decode(cursor))  
assert decoded["created_at"] == "2026-08-01T12:00:00"  
assert decoded["id"] == "uuid-123"
```



## Mock excesivo

| Pregunta | Señal en el código         |
|----------|----------------------------|
| P1       | Assert = mock.return_value |

```
def test_obtener_historial_service(mocker):  
    # Arrange  
    mock_repo = mocker.Mock()  
    mock_repo.buscar_por_comercio.return_value = [  
        {"id": "1", "monto": 100, "estado": "approved"},  
    ]  
    service = HistorialService(repository=mock_repo)  
  
    # Act  
    resultado = service.obtener_historial(comercio_id="c1",  
    filtros={}, cursor=None, page_size=50)  
  
    # Assert  
    assert resultado["data"] == [  
        {"id": "1", "monto": 100, "estado": "approved"},  
    ] # ⚠️ assera el mock, no la transformación del se
```

P1: el assert compara el output con exactamente lo que el mock devuelve · el service podría no hacer NADA y el test pasaría

Los tests de service deben validar transformaciones, no el eco del mock del repository. Si el service no transforma nada, no necesita test unitario."

```
# Assert · verifica que el SERVICE transforma (enmascara  
PAN según PRD §5 PCI-DSS)  
assert resultado["data"][0]["pan_last4"] == "9012"  
assert "pan" not in resultado["data"][0] # el PAN completo  
NO debe estar en la respuesta
```



## Happy path dominante

Pregunta

Señal en el código

P3

Todos los tests devuelven 200

```
# Suite completa de tests de un estudiante
```

Pytho

```
def test_get_historial_devuelve_200():
    response = client.get("/api/v1/transacciones?
desde=2026-07-01&hasta=2026-08-01", headers=valid_headers)
    assert response.status_code == 200

def test_get_historial_devuelve_data():
    response = client.get("/api/v1/transacciones?
desde=2026-07-01&hasta=2026-08-01", headers=valid_headers)
    assert "data" in response.json()

def test_get_historial_devuelve_pagination():
    response = client.get("/api/v1/transacciones?
desde=2026-07-01&hasta=2026-08-01", headers=valid_headers)
    assert "pagination" in response.json()
```

"Generá al menos 1 test por cada flujo del diagrama de secuencia, no solo el happy path. Del PRD, cada Regla de negocio debe tener 1 test que la valide."

P3: cero cobertura de las 2 reglas críticas del PRD (rango > 90 días · JWT inválido)

3 tests que verifican variaciones del mismo happy path · pero ninguno cubre el resto del diagrama de secuencia



## Happy path dominante

| Pregunta | Señal en el código            |
|----------|-------------------------------|
| P3       | Todos los tests devuelven 200 |

```
def test_historial_happy_path_devuelve_200():
    # Arrange · Act
    response = client.get(
        "/api/v1/transacciones?
        desde=2026-07-01&hasta=2026-08-01",
        headers={"Authorization": "Bearer valid_jwt"},
    )
    # Assert
    assert response.status_code == 200
    body = response.json()
    assert "data" in body and "pagination" in body
```

```
def test_historial_con_rango_mayor_90_dias_devuelve_400():
    # Arrange · Act
    response = client.get(
        "/api/v1/transacciones?
        desde=2026-01-01&hasta=2026-08-01", # 7 meses
        headers={"Authorization": "Bearer valid_jwt"},
    )
    # Assert · PRD §3 · §5
    assert response.status_code == 400
    assert "excede el máximo permitido" in response.json()
    ["error"]
```

```
def test_historial_sin_jwt_devuelve_401():
    # Arrange · Act
    response = client.get(
        "/api/v1/transacciones?
        desde=2026-07-01&hasta=2026-08-01",
    )
    # Assert · Diagrama de secuencia · flujo alt "JWT inválido"
    assert response.status_code == 401
```

## Mocks y fixtures

**El mock SUSTITUYE una dependencia · el fixture PREPARA el dato.**

### Ejemplos sobre GET /api/v1/transacciones

- **MOCK** · reemplaza el repositorio para que devuelva lo que vos decidís:

```
mock_repo.obtener.return_value = [] → el endpoint responde 200 sin tocar la DB
```

- **FIXTURE** · prepara el estado que el test necesita antes de ejecutarse:

```
@pytest.fixture
def tres_tx():
    return [Transaccion(monto=1500.00, ...)]
```

# Lab en vivo · Pasos 3 y 4

Implementación + Verificación con TDD asistido

## Recap · dónde estamos en el TDD



### Estado ideal para hoy

- Modelo Pydantic escrito en `app/schemas/transaccion.py`
- 3 tests escritos en `tests/test_historial.py`
- **pytest corriendo con TESTS EN ROJO**

## PASO 3 · Implementación (rojo → verde)

### Regla del Paso 3

**Escribí SOLO el código mínimo para que los tests pasen.**

*Sin funcionalidades extras · sin optimizaciones · sin scope creep. Los tests son el freno.*

#### Arquitectura · 3 capas

Router → Service → Repository. El router NO habla con la DB.

#### Validación ≠ Regla de negocio

Pydantic valida formato. El invariante del dominio va en Service.

#### 1 test rojo a la vez

Implementá para pasar UN test. Confirmá verde. Pasá al siguiente.



## PROMPT-C · Implementación mínima (Paso 3)

```
#file:docs/prd/PRD.md
```

```
#file:app/schemas/transaccion.py
```

```
#file:tests/test_historial.py
```

### CONTEXTO:

Stack: Python 3.12 · FastAPI · Pydantic v2 · pytest

Clean Architecture · 3 capas: routers/ · services/ · repositories/

Los 3 tests de test\_historial.py están en ROJO. Necesito implementación mínima.

### TAREA:

Generará el mínimo código para que los 3 tests pasen:

- app/routers/historial.py con GET /api/v1/transacciones
- app/services/historial\_service.py con la lógica de filtros + cursor
- app/repositories/historial\_repo.py con fake in-memory (SIN Postgres real hoy)

### REQUISITOS:

- Router solo maneja HTTP (validaciones · códigos · headers)
- Service contiene reglas de negocio (rango > 90 días · enmascarar PAN)
- Repository es fake in-memory · 5 transacciones de ejemplo · fijas
- Sin dependencias nuevas (no instalar nada)

RESTRICCIONES: solo Python · sin datos reales · determinístico



## Antes de aceptar el código · 5 preguntas rápidas

1

¿La estructura respeta las 3 capas?

*Router NO importa el repository. Service NO importa FastAPI.*

2

¿La regla de negocio vive en el Service (no en Pydantic)?

*Ej: 'rango <= 90 días' es Pydantic. 'usuario solo ve sus transacciones' es Service.*

3

¿Instaló dependencias que no pediste?

*email-validator · SQLAlchemy · descartalas si no las pediste.*

4

¿Inventó campos que no están en el PRD ni en tu schema?

*Si tu Pydantic tiene 5 campos y el código usa 7 · alguien inventó.*

5

¿Los tests están verdes por lógica o por casualidad?

*Ejecutá pytest -v y lee cada assert · un verde sin auditoría es cosmético.*

## ✓ PASO 4 · Verificación (pantalla VERDE)

***Verde no es éxito · es una hipótesis validada.***

El paso 4 verifica que los tests pasan Y que los asserts son efectivos.

### Checklist de verificación · en el terminal

# 1. Ejecutar tests con verbosidad

```
uv run --frozen pytest -v
```

# 2. Confirmar cobertura mínima

```
uv run --frozen pytest --cov=app --cov-report=term-missing
```

# 3. Buscar señales de alarma

```
grep -rn "assert True" tests/
```

```
grep -rn "try:" tests/      # ¿por qué hay try/except en un test?
```

# 4. Linter y tipos

```
uv run --frozen ruff check .
```

```
uv run --frozen mypy app/
```

## PROMPT-D · Auto-crítica del código (Paso 4)

```
#file:app/routers/historial.py
#file:app/services/historial_service.py
#file:app/repositories/historial_repo.py
#file:tests/test_historial.py
```

### CONTEXTO:

Acabo de implementar el endpoint GET /api/v1/transacciones con TDD asistido.  
Los 3 tests pasan en verde.

### TAREA:

Actúa como auditor de código. Busca específicamente:

1. Si algún assert es tautológico o compara contra mock.return\_value
2. Si el Service tiene lógica que debería estar en el Repository (o al revés)
3. Si hay validaciones duplicadas entre Pydantic y Service
4. Si hay imports o funciones que no se usan
5. Si algún test cubre solo el happy path

### Devolvéme:

- Lista de hallazgos con severidad (alta · media · baja)
- Sugerencia concreta de corrección por cada hallazgo
- SIN escribir código todavía · solo auditoría

RESTRICCIONES: no modifiques archivos · solo reportá.



## Casos borde que vamos a buscar

|              |                                                                                        |
|--------------|----------------------------------------------------------------------------------------|
| Nulos        | ¿Qué pasa si comercio_id llega vacío? · ¿Y si el JWT llega incompleto?                 |
| Negativos    | ¿page_size = -1? · ¿desde > hasta? · ¿monto negativo?                                  |
| Concurrencia | Si 2 requests llegan simultáneos con el mismo cursor, ¿la respuesta es determinística? |
| Tiempos      | ¿Qué pasa con timezone? · ¿timestamps duplicados exactos?                              |



*No hace falta cubrir TODOS hoy. Elegí 2 casos borde y agregalos como tests explícitos.*

# Cierre · Gobernanza de tests con IA

3 principios que sostienen la calidad de tu código asistido



## Gobernanza de tests · qué significa

**Las reglas verificables que deciden cuándo un test asistido es aceptable.**

*No es burocracia · es trazabilidad, responsabilidad humana y criterios de aceptación explícitos.*

### Qué gobernás en la práctica

- **Trazabilidad** · qué modelo generó qué archivo, registrado en AI\_USAGE.md.
- **Responsabilidad humana** · el verde lo firma una persona, no el modelo.
- **Criterios explícitos** · las 5 preguntas de auditoría y el checklist de casos borde.
- **Higiene de datos** · fixtures sintéticos y deterministas · sin PII en el repositorio.

*Anclaje académico: EU AI Act Art. 14 (human oversight) · NIST AI RMF · funciones Govern y Measure.*

## Principio 1 · Cross-model auditing

**No uses el MISMO modelo para código + tests + validación.**

*Un modelo evalúa mal sus propios errores · tiene el mismo sesgo que tuvo al generarlos.*

### **Cómo aplicarlo hoy · barato y efectivo**

- Usá Copilot (GPT) para generar código · Claude para auditarlo (o al revés)
- Si solo tenés un modelo · pedile explícitamente que audite "como si otro modelo lo hubiera escrito"
- Para tests críticos: escribí al menos 1 test a mano, no lo generes con IA
- Documentá en AI\_USAGE.md qué modelo generó qué archivo

*Anclaje académico: EU AI Act Art. 14 · human oversight en sistemas de alto riesgo.*



## Principio 2 · Verificación manual de casos borde

### Los casos borde son responsabilidad TUYA

*La IA optimiza para el happy path porque su corpus lo tiene sobre-representado.*

#### Checklist obligatorio antes de aceptar cualquier suite de tests

- ☐ Nulos · ¿qué pasa con inputs vacíos o None?
- ☐ Negativos · ¿números negativos · fechas invertidas · rangos imposibles?
- ☐ Concurrencia · ¿2 requests simultáneos rompen algo?
- ☐ Tiempos · ¿timezone · DST · timestamps duplicados exactos?

*Anclaje académico: NIST AI RMF · función Measure · testeo de robustez y edge cases.*

## Principio 3 · Higiene de datos

**Nunca commites datos reales al repo · ni siquiera 'para probar'.**

*Los tests deben usar fixtures sintéticos · deterministas · sin PII.*

### Aplicación práctica al Lab 3

- PANs de tarjeta: siempre usar números de prueba (4532 XXXX XXXX 9012)
- Emails: usar dominios reservados (@example.com · @test.local)
- .env NUNCA se commita · sí .env.example con valores dummy
- Antes del push · `git diff staged | grep -iE '(password|secret|key|token)'`

*Anclaje académico: EU AI Act Art. 12 · registros técnicos + PCI-DSS + GDPR.*



## Cierre · qué te llevás hoy

### 1 idea que sostener

***"El verde de un test es una hipótesis. La auditoría es lo que lo convierte en prueba."***

3 principios de gobernanza: cross-model auditing · casos borde manuales · higiene de datos.



### Próxima sesión · jue 13-ago · Arranque M4 · Agentes

- Cambia el paradigma · pasamos del chatbot al agente autónomo
- Introducimos MCP (Model Context Protocol) · el estándar que conecta agentes con tools
- Preview Lab M4 · vas a construir tu propio agente

*Lab 3 Cierra el 16 de Agosto 23:59h · si te trabás avisá en Discord*